The Database: How Data is Stored and Protected

A plain-English guide to how Supabase stores information, connects campus food shops, keeps money safe, and updates kitchen screens in real time.

What Supabase Actually Stores

Here are the real tables active inside V FOODS's database, with a simple explanation of what each one holds:

- **outlets**: Holds the list of campus canteens, food courts, and festival stalls, including their names, campus locations, and whether they are currently open or closed.
- **menu_items**: Holds all the dishes and drinks available to buy, along with their prices, food category, veg or non-veg badge, time availability, and current stock count.
- **orders**: Holds every food order placed by students, recording the four-digit token number, current status (such as Placed, Preparing, Ready, or Collected), shop payout, and final total.
- **order_items**: Holds the individual food dishes and quantities inside each order, so the kitchen knows exactly which items to cook and pack.
- **profiles**: Holds basic user accounts across the campus, storing each person's full name, role (student, staff, shop owner, or admin), phone number, and account status.
- **wallets**: Holds each student's current prepaid balance so they can pay for food instantly without needing bank cards or internet banking at the counter.
- **wallet_txns**: Holds an unchangeable history of every rupee added or spent, recording the exact reason, time, and reference code for every balance change.
- **payments**: Holds all Paytm payment attempts across both wallet top-up and direct order checkout, storing the Paytm order ID, bank transaction ID, payment purpose, method, and status.
- **outlet_paytm_accounts**: Holds each shop's Paytm recipient sub-account ID, onboarding status, and configurable shop split percentage (default 90%).
- **platform_settlement_account**: Holds the startup's master Paytm settlement account and fixed platform split percentage (5%).
- **college_settlement_accounts**: Holds the host university's active campus Paytm account and institutional split percentage (5%).
- **payment_splits**: Records the exact three-way revenue distribution (Shop, Platform, College) generated automatically on every meal order.
- **refunds**: Holds full audit records of customer refunds across orders, wallet top-ups, and split rollbacks.
- **audit_logs**: Holds an immutable forensic audit log recording all staff menu price changes, inventory adjustments, and administrative overrides.

- **invites**: Holds special single-use invite codes used by administrators to add new shop owners and kitchen staff members safely.
- **settings**: Holds campus-wide master controls, such as turning on festival mode to feature special event food stalls.

> *For the complete relational schema, full column definitions, and constraint specifications, see the technical reference in DATABASE.md.*

How Shops Are Connected

Every food stall on campus has a short unique code called `outlet_id` (such as `gazebo_c1`, `dakshin_chitra`, or `north_sq_c2`).

Think of `outlet_id` as an invisible thread that connects everything belonging to that one shop: - **Menu Items**: Every dish is tagged with an `outlet_id`, so a roll from Gazebo C1 never accidentally shows up in the North Square kitchen. - **Staff Accounts**: Every cook and cashier has an `outlet_id` saved on their profile, which locks their screen strictly to their own counter. - **Orders**: When a student places an order, the database tags it with the shop's `outlet_id`, directing the order straight to that shop's kitchen tablet.

Because of this single thread, different shops can operate on the same platform at the same time without their food tickets, staff, or earnings ever mixing together.

How the Four Roles See Different Things

V FOODS uses a built-in database security system called **Row-Level Security (RLS)**.

In most simple websites, the server sends all information to the phone, and the app just hides buttons the user shouldn't click. If a clever person inspects the network, they can see other people's data.

In V FOODS, security is handled inside the database itself: - **The Student Rule**: A student's phone can only read rows that have their own user ID. The database physically refuses to return another student's wallet balance or orders, no matter what request is sent. - **The Kitchen Staff Rule**: Kitchen staff can only read orders and menu items that have their shop's `outlet_id`. A staff member at Gazebo cannot see orders placed at North Square. - **The Shop Owner Rule**: Shop owners can see their own outlet's daily revenue, staff team, and menu prices, but cannot see any other canteen's sales. - **The Super Admin Rule**: Campus administrators have platform-wide access to view all canteens, monitor overall campus revenue, and onboard new food stalls.

The database itself blocks unauthorized access at the root, making data leaks physically impossible.

The Order Journey, Step by Step

Here is what happens inside the database from the moment a student taps "Place Order" to when they pick up their food:

1. **Safety Checks**: The student taps "Place Order". A secure database procedure called `place_order_wallet` runs inside one locked transaction. It confirms that the canteen is currently open, every selected item is in stock, and the student's wallet has enough balance.
2. **Instant Wallet Deduction**: The database subtracts the total cost from the student's wallet and immediately creates an unchangeable record in `wallet_txns`. If the balance is even one rupee short, the whole process stops with zero changes made.
3. **Stock Auto-Decrease**: The database automatically decreases the stock count of each ordered dish. If an item runs down to zero portions left, the database automatically flips its status to "sold out" so nobody else can order it.
4. **Kitchen Ticket Created**: The database assigns a fresh 4-digit pickup token (like `142`) and inserts the new order row into the `orders` table.
5. **Realtime Screen Update**: Supabase Realtime immediately beams the new order over a live WebSocket connection straight to the kitchen's tablet. The tablet chimes, and the ticket appears under "Incoming Orders" within a fraction of a second, without anyone needing to refresh the page.
6. **Milestone Progression**: As the kitchen cooks the food, staff tap buttons to advance the status:
7. Staff tap **Start Preparing** -> Database status changes to `preparing`.
8. Staff tap **Mark Ready** -> Database status changes to `ready`. This triggers an instant SMS or WhatsApp notification to the student with their token number.
9. **Collection & Handover**: The student arrives at the counter and shows their token or QR pass. Staff tap **Mark Collected** -> Database updates the status to `collected`, completing the order journey.

The Money Rules

V FOODS follows two strict money rules to protect both students and shop owners:

1. The Database Always Calculates Prices Itself

- A student's phone or browser is never allowed to tell the database how much an order costs.
- When an order is placed, the phone only sends the item IDs and quantities (for example: "2 samosas").
- The database looks up the official menu prices saved in its own tables, multiplies them, and computes the exact total.
- **Why this matters**: A malicious user could tamper with a phone app to claim an item costs 1 rupee instead of 80 rupees. Because the database looks up official prices itself, price tampering is impossible.

2. Three-Way Split (Shop, Platform, College) Is Calculated Automatically

- When an order is paid—whether via direct Paytm checkout or campus wallet—the database immediately calculates and records an unalterable three-way revenue distribution in `payment_splits`:

- **Shop Share**: The fulfilling outlet's configured split share (default **90%**, customizable per outlet agreement).
- **Platform Share**: The V Foods startup company's fixed app-wide platform fee (**5%**).
- **College Share**: The host institution's active campus royalty (**5%**).
- A database trigger guarantees that Shop + Platform + College splits strictly sum to 100%, and exact penny calculations ensure zero rounding leakage.
- All split logic is executed inside database stored procedures (`calculate_order_split`) and cannot be altered or bypassed by any app screen.
